Decision Making & Repetition

L03 - Decision Marking and Repetition

Intended Learning Outcomes

After completing the lesson, students should be able to:

- Control program flows using if-else and switch statements
- Apply logical operators for decision making
- Create program loops using while loop, do-while loop and for loop.
- Apply program control statements inside a repetition structure

Decision Making statements

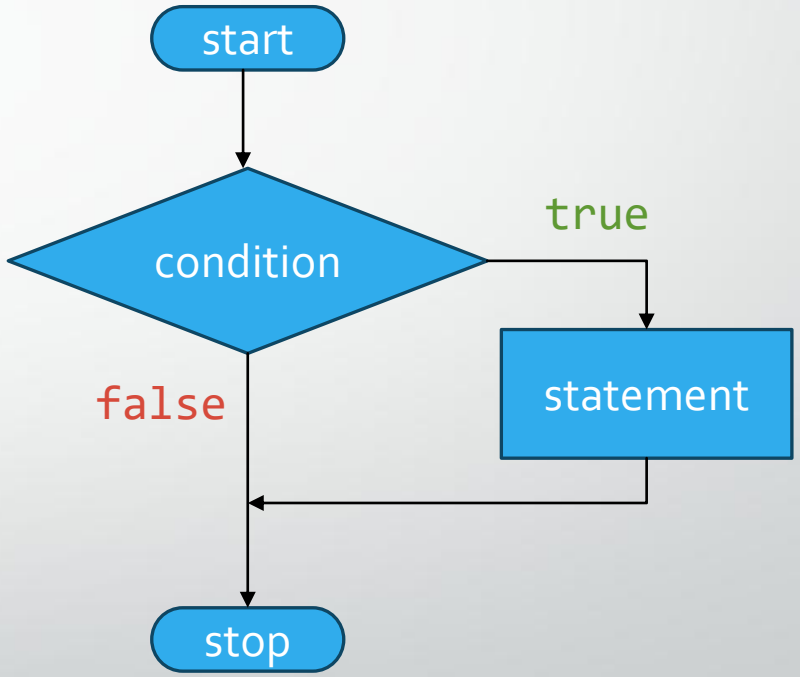
if statement

- Decide if the next statement / block is executed or not
  - Depends on the Boolean expression result
- Syntax

Round bracket () is a must.

```
if (condition){
    statement;
}
```

Curly brace is optional if only 1 statement
[Suggest to use for consistence.]



Programming Style

- Line break does not change the meaning

```
if (mark >=50) grade='P';
```

=

```
if (mark >=50)
    grade='P';
```

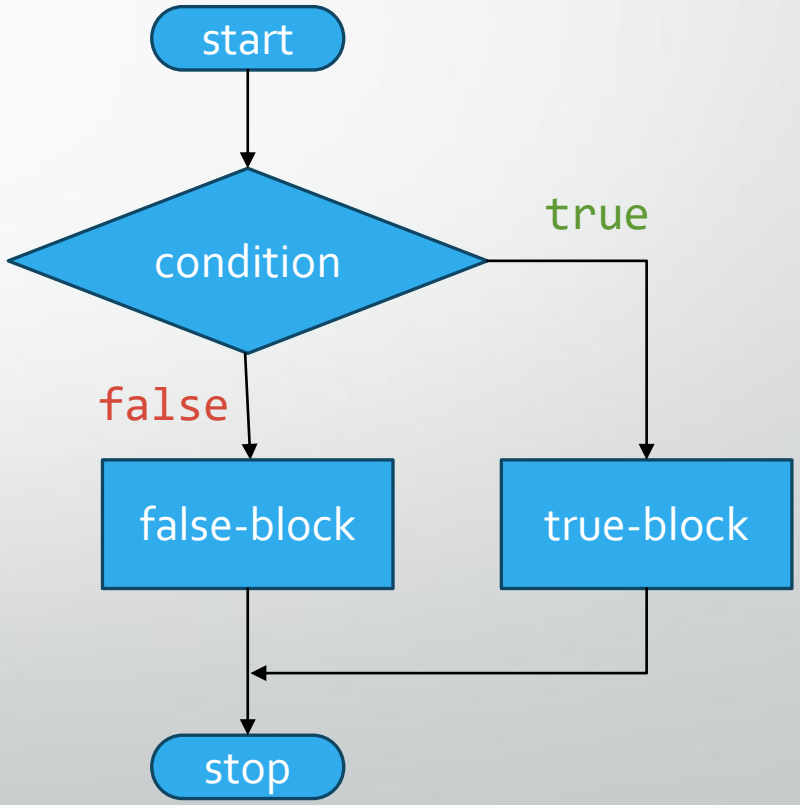
https://www.w3schools.com/java/java_conditions.asp

if-else statement

- Decide which statement / block to be executed
  - Depends on the Boolean expression result
- Syntax

Round bracket () is a must.

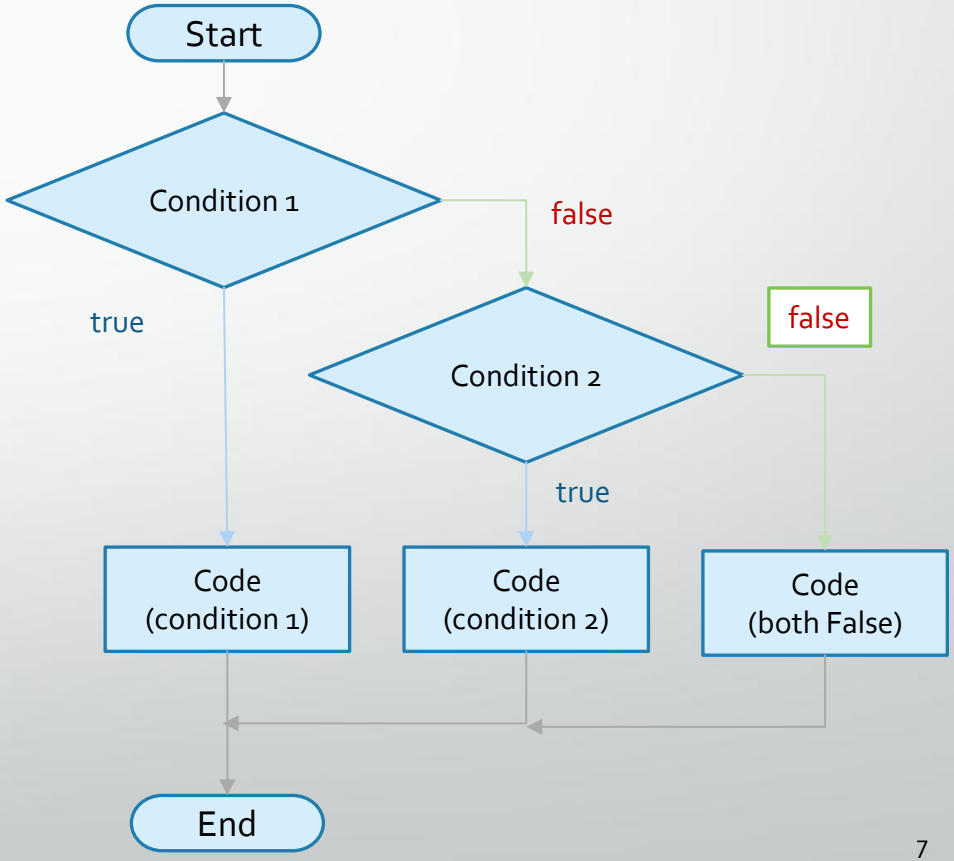
```
if (condition){
   true_block;
}else{
   false block;
}
```



Multiple if-else statements

- Syntax:

```
if (condition 1)
    code when condition 1 is true
else if (condition 2)
    code when condition 2 is true
else
    code when both were false
```



Other branching statements

- Condition operator
- Switch statement

Conditional Operator

- a ternary (three operand) operator.
- can be used to write a simple `if-else` like statement.
  - commonly used to supply a value to a variable
- Syntax:

```
BooleanExpression ? ValueTrue : ValueFalse
```

- If *`BooleanExpression`* is true, value of expression is *`ValueTrue`*.
- Otherwise, value of the expression is *`ValueFalse`*.

Conditional Operator - Example

- Example:

```
result = x > y ? "Greater" : "Not";
```

- This line is functionally equivalent to:

```
if(x > y)
  result = "Greater";
else
  result = "Not";
```

switch statement

- a control flow statement
  - select one of many code blocks to execute
  - based on the value of a variable or expression.
- an alternative to using multiple if-else
  - when multiple conditions to check.
- evaluate an *integer* type or *character* type variable

Reference: https://www.w3schools.com/java/java_switch.asp

switch statement

```
switch (Expression) {
    case value1:
        // code to be executed if variable == value1
        break;
    case value2:
        // code to be executed if variable == value2
        break;
    // ...
    default:
        // code to be executed if all doesn't match
        break;
}
```

`switch (Expression)` clause

- `Expression`
  - result of expression is matched with the case values
  - can be a byte, short, int, long, or char. (Since Java 7, also be a String)
- If there is an associated case statement that matches that value,
- program execution will be transferred to that case statement.

`case valueX:` clause

- Each `case value` must be unique.

```
case valueX:
    // place one or more statements here
break;
```

- If `Expression` matches the `valueX`,
  - will executed statements between the colon and the `break` statement.

`break` and `default`

- `break`
  - It ends the `case` statement.
  - optional
  - If a `case` does not contain a `break`, execution continues into the next `case`.

- `default:`
- will be executed if no case value matches the *Expression*
- Optional
- Like 'else' clause in if-else statement

Switch Example

```
String answer;
java.util.Scanner scan = new Scanner(System.in);
System.out.print("Are you HKBU student? ");
answer = scan.nextLine();
switch (answer.toLowerCase()){
    case "yes":
        System.out.println("Good.");
    case "i am":
        System.out.println("Hi, me too.");
        break;
    case "no":
        System.out.println("Hi, I come from HKBU.");
        break;
    default:
        System.out.println("I don't understand.");
}
scan.close();
```

Comparison and Logical operators

Equality & Relational operators

- Compare the values of two expressions
- Result would be true or false only
- Floating-point equal (==) may not be exact
- Object equal (==) compares reference
  - Not content of variable

Name	Operator	Example
Equal	==	x == y
Not equal	!=	x != y
Greater than	>	x > y
Less than	<	x < y
Greater than or equal	>=	x >= y
Less than or equal	<=	x <= y

Floating-point equality check

- Equality (==) check and Inequality (!=)
  - may have unexpected result
  - Due to rounding
  - 2.00001 == 2.0?
- For floating-point figures

```
double num1 = 0.2999999 *2.99999;
double num2 = 0.3 * 3;
double tolerance = 0.00001; // Set a small tolerance value

if (Math.abs(num1 - num2) < tolerance) {
    System.out.println("They are considered equal.");
} else {
    System.out.println("They are not equal.");
}
```

Character comparison

- char variables are stored as Unicode
  - Ordinal: has order
- Example:

```
boolean a_before_c = 'a' < 'c'; // true
boolean A_after_a = 'A' > 'a';  // false
// as 'A' (\u0041) < 'a' (\U0061)
```

Conditional operators

Operator	meaning	Description
&&	AND	Return true if both statements are true
||	OR	Return true if one of the statements is true
!	NOT	Reverse the result, return false if the result is true

A	B	A && B	A || B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

A	!A
true	false
false	true

Short Circuiting

- Logical AND (&&)
  - will evaluate to false
  - as soon as one of its operands is false
- Logical OR (||)
  - will evaluate to true
  - as soon as one of its operands is true

```
int num1 = 5;
int num2 = 10;
// Short-circuit behavior of &&
if (num1 < 0 && ++num2 < 0) {
    System.out.println("Skip.");
}
// Short-circuit behavior of ||
if (num1 > 0 || num2++ > 0) {
    System.out.println("num1 > 0 short-circuit.");
}
```

Boolean Variable

- A variable with boolean data type
  - Can store true or false value
- Boolean expression
  - A calculation that results in true or false

```
int age = 30;
boolean retire_age = age >= 65;
if (retire_age)
    System.out.println("Time to retire.");
```

Exercise: Grade Calculator

- Write a Java program that takes a student's score as input and prints out the corresponding grade based on the following criteria:
- Instructions:
  - `Prompt the user to enter a score between 0 and 100.`
  - `Use if-else statements to determine the grade based on the score.`
  - `Print the grade to the console.`

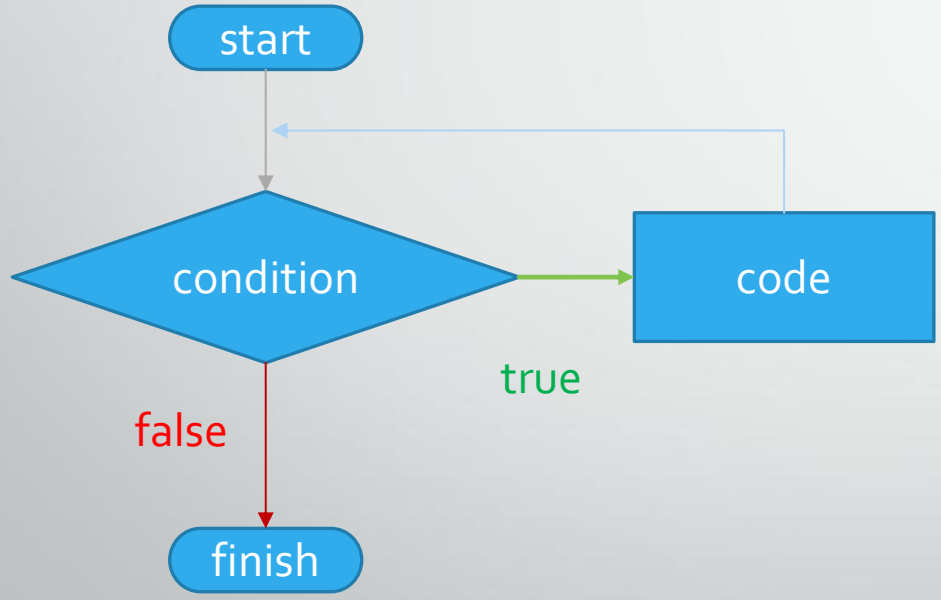
Score	Grade
Greater than or equal to 80	A
Between 70 and 80 (exclusive)	B
Between 60 and 70 (exclusive)	C
Between 50 and 60 (exclusive)	D
Less than 50	F

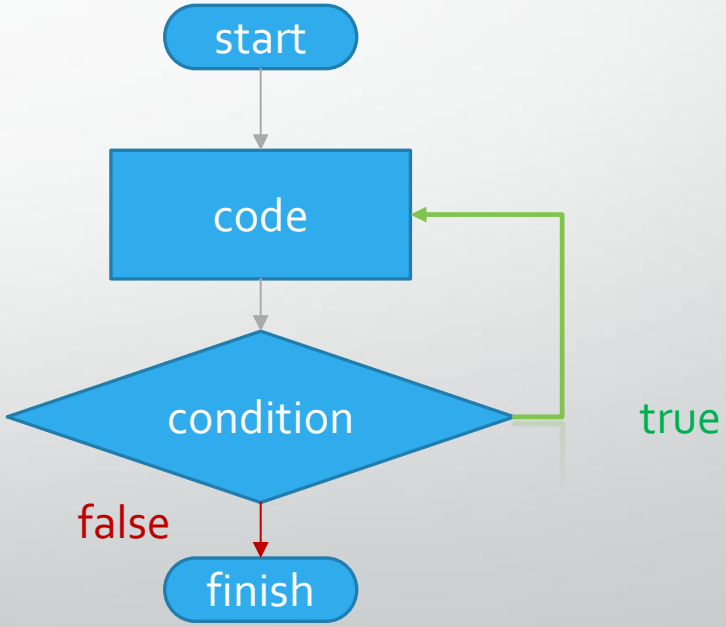
Repetition Structures

While loop, do-while loop, for loop

Loop

- a sequence of instructions that is continually repeated until a certain condition is reached or a condition is declared false



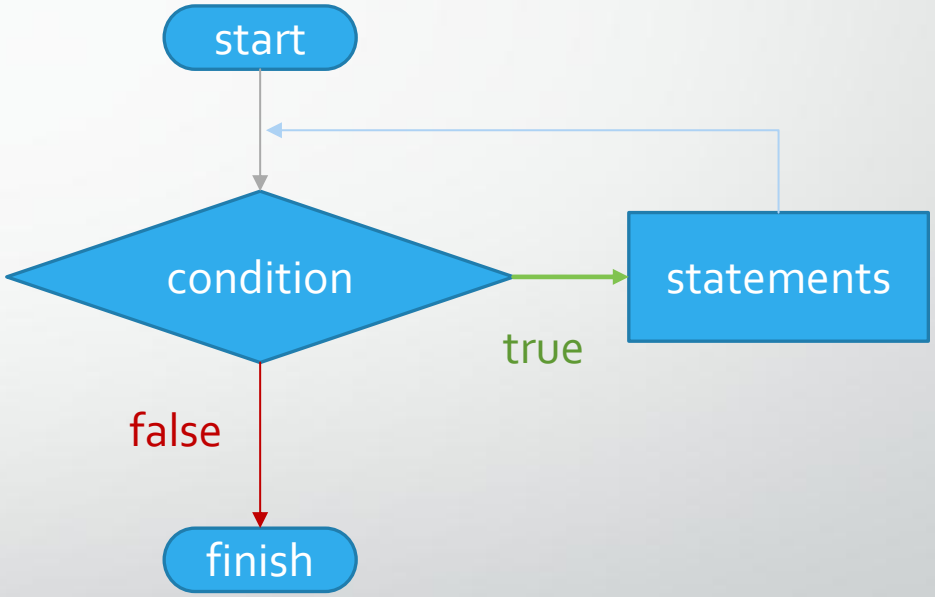


Reference: https://computersciencewiki.org/index.php/Loops

While loop

- Execute a set of statements as long as condition is true
- Syntax:

```
while (condition){
    statements_in_loop
}
```



- Note:
  - Condition evaluates to true or false

Source: https://www.w3schools.com/java/java_while_loop.asp

Infinite loop

- If the condition never reach to false, the loop will repeat forever
- To break the execute of infinite loop,
  - press Control-C during execution
- Examples

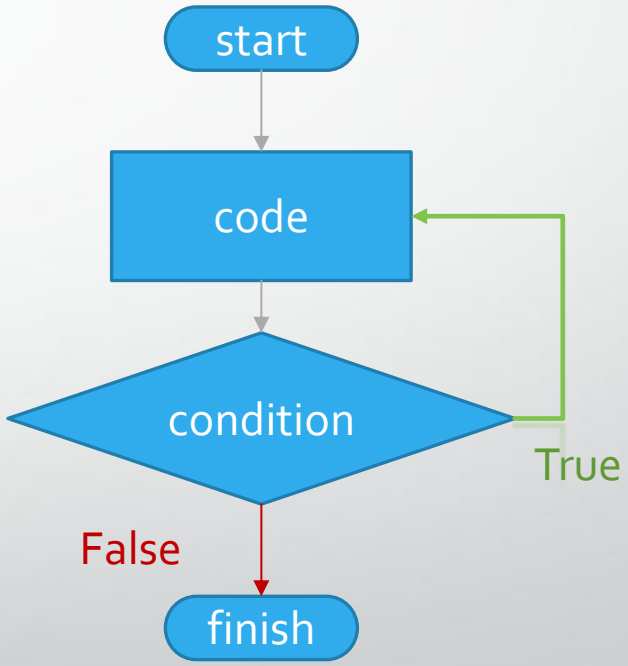
```
int i = 0;
while (i < 5) {
    System.out.println("i=" + i);
    i = i - 1;
}
System.out.println("End of loop.");
```

```
int i = 0;
while (true) {
    System.out.println("wait: " + i);
    i = i + 1;
}
//System.out.println("End of loop.");
```

Do while loop

- Execute the code before checking condition
  - Execute block for at least once
- Syntax:

```
do {
    code;
} while (condition);
```



Example - DoWhileLoop.java

```
do {
    System.out.print("Enter Y/N: ");
    key = scanner.nextLine();

    if (!key.matches("[yYnN]")) {
        System.out.println("Wrong input.");
    }
} while (!key.matches("[yYnN]"));
```

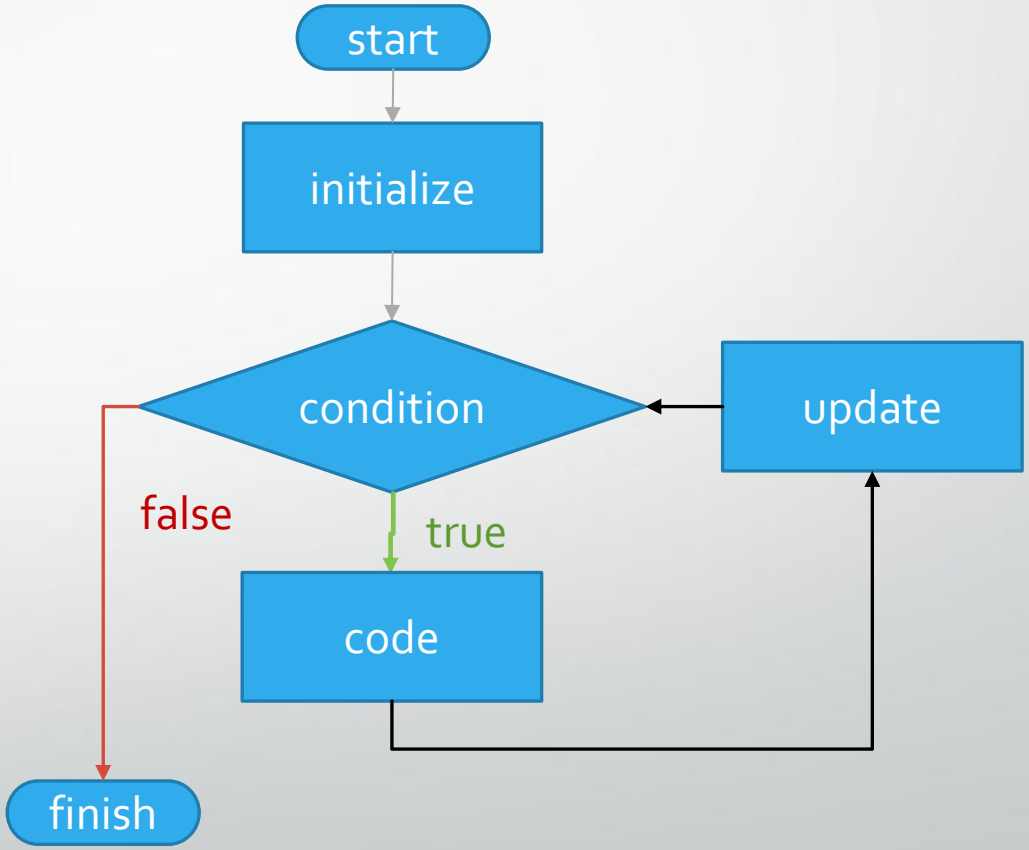
for loop

- Usually iterating over a set of numbers
- Syntax

```
for (initialize; condition; update){
    code_in_loop;
}
```

- Example

```
for (int i=0; i<5; i++){
    System.out.print(i);
}
```



For-each loop (enhanced for loop)

- To iterate over elements of an array or collection
  - without worrying about the index or length.
- Introduced in Java 5
- Syntax:

```
for (Type element : collection) {
    // code to be executed for each element
}
```

For-each loop Example

Integer array. More in "Arrays and Collections"

```
int[] numbers = {1, 2, 3, 4, 5};
for (int num : numbers) {
    System.out.println(num);
}
```

Java Array

- Store multiple values in a single variable
- Declare the variable type with square brackets ([])
- Can be initialized with curly brace {} and value list when declared
- Example

```
String[] names;
int[] ages = {18, 19, 27};
```

https://www.w3schools.com/java/java_arrays.asp

Using array

- array elements can be assessed using [index] notation
  - Index is zero-based
  - `[0]` is the first element. `[1]` is the second element, etc.)
- Length property returns number of elements in the array
- May get Index ArrayIndexOutOfBoundsException exception
  - Access elements outside value range `[0, length-1]`

Array Example

```
String [] names;
int[] ages = {18, 19, 25};
names = new String[3];
names[0] = "John";
names[1] = "Merry";
names[2] = "Ben";
for (int i=0; i < names.length; i++){
    System.out.println(names[i] + " is " + ages[i] + " years old.");
}//
```

Exercise – Sum from 1 to n

Requirement

- Calculate the sum from 1 to n.
- Hints:
  - Use a sum variable to store the total.
  - Use a while loop to count from 1 to n.

$$sum = \sum_{x=1}^{n} x$$

Sample output

```
Enter n: 10

Sum from 1 to 10 is 55.
```

```
Enter n: 20

Sum from 1 to 20 is 210.
```

Program control

Break statement

- To stop the loop before it has looped through the sequence.
- Usually used inside an if-statement
- Example

```
String[] zoo = {"dog", "cat", "rabbit", "monkey"};

for (String animal : zoo) {
    if (animal.equals("rabbit")) {
        break;
    }
    System.out.println("Animal is " + animal);
}
```

Continue statement

- Stop the current iteration of the loop, and continue with the next
- Usually used inside an if-statement
- Example

```
String[] zoo = {"dog", "cat", "rabbit", "monkey"};

for (String animal : zoo) {
    if (animal.equals("rabbit")) {
        continue;
    }
    System.out.println("Animal is " + animal);
}
```

```
Animal is dog
Animal is cat
Animal is monkey
End of loop.
```

Exercise – Skip 7

Requirements

- Ask the user for a positive integer (n).
- List all the position numbers excluding those that are multiple of 7 or last digit is 7. The list should end at n.

Sample output

```
Enter n: 30
1, 2, 3, 4, 5, 6, skip
8, 9, 10, 11, 12, 13, skip
15, 16, skip
18, 19, 20, skip
22, 23, 24, 25, 26, skip
skip
29, 30,
```

Summary

BRANCHING
(IF-ELSE)

LOOPING (FOR,
WHILE, DO-WHILE)

PROGRAM CONTROL
(BREAK, CONTINUE)